JAVA · KOTLIN / SPRING

신지섭
백엔드 포트폴리오

Java·Kotlin 백엔드 개발자

이벤트 추적과 지연 분석 · 장애 시 환불 판단 · 동시 구매의 커넥션 대기 해결
ESD 플랫폼 / SNG / JobMate

github.com/tlswltjq · wltjq1203@icloud.com
2026.10.23

목차 · 대표 경험

세 프로젝트에서 해결한 문제

프로젝트	대표 문제와 변경	담당 · 기술	바로 가기
ESD	끊어진 이벤트 추적을 연결해 발행 대기를 줄였습니다. 장애 실험에서 잘못된 환불 원인을 찾아 판단 조건을 수정했습니다.	개인 프로젝트 Java · Spring Boot · Kafka	03–10
SNG	동시 구매가 추가 DB 커넥션을 기다리는 원인을 찾았습니다. 초기화를 주문 트랜잭션에 통합하고 실제 MySQL에서 검증했습니다.	개인 프로젝트 Kotlin · Spring Boot · JPA	11–14
JobMate	업로드 완료 전 뉴스 제출 시 영상 조회가 실패했습니다. 영상 레코드를 먼저 만들고, 변환 결과물의 재입력을 차단했습니다.	팀 5명 · 영상 파이프라인 담당 Java · S3 · Kafka · Python	15–16

주요 기술

Java · Kotlin · Spring Boot · JPA/Hibernate · MySQL · Apache Kafka · Docker Compose · JUnit 5 · Testcontainers · k6

구현·검증 자료

코드·테스트·측정 기록 17쪽
대표 구현과 다음 작업 18쪽

ESD · 8–9월 구현 개요

게임 등록부터 구매·다운로드·정산까지 구현

8–9월에는 스튜디오·스토어·다운로드·정산 등 9개 서비스를 이벤트로 연결했습니다. 10월에는 인증부터 구매·다운로드·정산까지 11개 모듈의 MVP를 재작성했습니다. 이 장의 장애·성능 실험과 환불 데모는 8–9월 구현의 결과입니다.

처리 구간	서비스	주요 책임
등록·출시	studio · review · catalog	파일 검증, 심사 이력, 공개할 빌드와 상품 정보 확정
조회·구매	store · order · payment	검색용 조회 모델, 서버 가격 계산, 결제·환불 처리
이용·정산	license · download · settlement	라이선스 지급·회수, 다운로드 권한, 매출·환불 원장



서비스별 DB를 두면서 결제·지급·정산을 하나의 트랜잭션으로 끝낼 수 없었습니다. 업무 데이터와 이벤트를 **같은 트랜잭션에 저장**한 뒤 Outbox로 전달하고, 소비 서비스에서 중복을 확인합니다. 결제는 성공했지만 지급은 실패한 상황의 환불·재처리 조건을 검증했습니다.

Java 21 · Spring Boot 3.5 · MySQL · Kafka · Redis · Elasticsearch · MongoDB · Docker

개발 과제와 서비스 구성 · 설계 결정 기록 · Outbox 구현

ESD · API 데모

실제 파일 다운로드·환불 후 권한 회수·정산 대사 확인

새 환경에서 가입·등록·심사·출시·구매·환불·월 마감을 두 번 실행했습니다. 회차마다 계정·상품·주문을 새로 만들고, 구매한 게임의 ZIP을 내려받아 업로드한 원본과 비교했습니다.

단계	검증한 동작	실행 결과에 남기는 값
등록·출시	실제 ZIP 업로드·검증, 심사 승인, 선택한 빌드 공개	buildId · releaseId · SHA-256
구매·이용	주문 금액과 승인액 일치, 라이선스 지급, 실제 ZIP 다운로드	orderNo · 파일 해시 일치
환불·정산	다운로드 권한 회수, 원장 역분개, 할인 부담 주체별 정산·월 대사	정산 금액 · 대사 결과 · CSV

원본 ZIP 일치

서명 URL에서 받은 파일과
업로드 파일의 SHA-256 대조

다운로드 권한 회수

환불한 주문의 라이선스와
다운로드 권한 회수 확인

월 정산 대사 일치

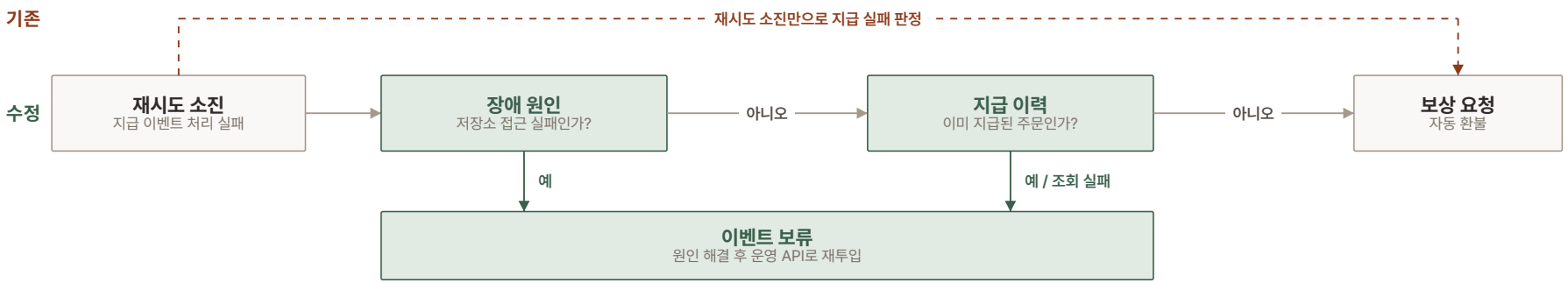
할인 부담·환불 조정액 반영,
대사 균형과 CSV 내역 확인

2026.09.30 · 소스 `7873593` · 빈 DB·객체 저장소 볼륨 · 회차별 57개 검증과 상태 검사 16개 통과. 공개 API와 결제 시뮬레이터로 검증했습니다. 외부 PG와 상점 웹 화면은 검증 범위에서 제외했습니다.

실행 절차·회차별 기록 · `bash scripts/api-demo.sh run` → summary.md / run-1.json / run-2.json

장애 범위를 나누자 잘못된 환불 경로가 드러남

스키마 전체 차단에서는 환불이 없었지만, 라이선스 테이블만 60초 차단하자 정상 결제 8건이 환불됐습니다. 전체 차단에서는 Outbox 저장도 실패해 문제가 가려졌습니다. 테이블만 차단하면 보상 이벤트는 저장됐고, 재시도 소진을 지급 실패로 판단한 결과가 환불로 이어졌습니다.



환불 전에 **실패 원인과 실제 지급 이력**을 확인하도록 바꿨습니다. 저장소 장애이거나 이미 지급된 주문은 보류합니다. 지급 이력 조회 자체가 실패한 경우도 보류합니다. 확인 가능한 지급 실패만 보상 경로로 전달합니다.

KafkaErrorHandlerConfig · D-027·D-028 재현 기록

같은 장애 조건에서 오환불 8건을 0건으로 줄임

주문 40건을 초당 2건씩 보내고 라이선스 테이블 접근을 60초 차단했습니다. 수정 후 같은 조건으로 다시 실행했습니다.

처리 결과	수정 전	수정 후
정상 지급	32건	33건
보류	0건	7건
오환불	8건	**0건**

재투입 결과

장애 해소 후 보류된 7건을 재투입했습니다. 재투입 요청 후 **1초 이내에 7건 모두 지급**됐습니다.

장애 주입 조건·결과 · 환불·보류 조건 테스트

재시도 시간을 늘리는 방법도 검토했습니다

블로킹 재시도를 길게 유지하면 같은 파티션의 후속 주문도 기다립니다. 보류 후 재처리하도록 해 장애가 난 메시지 때문에 정상 메시지가 계속 대기하는 상황을 줄였습니다.

재투입은 원인 해결 후 실행합니다

보류 이벤트를 자동으로 다시 넣으면 같은 오류가 반복될 수 있습니다. 운영 API로 재투입하고 지급 결과를 확인하도록 했습니다. 자동 환불 범위를 줄인 만큼 보류 건 확인과 재처리가 필요합니다.

ESD · 사례 2 / 원인과 수정

재시도 중인 이벤트보다 같은 주문의 후행 이벤트가 먼저 발행됨

결제 완료 이벤트의 Kafka 발행이 실패한 뒤, 같은 주문의 결제 취소 이벤트가 먼저 발행됐습니다. 메시지 키를 주문번호로 지정해도 **Kafka에 보내기 전에 바뀐 발행 순서**는 복구되지 않았습니다.

기존: 실패한 이벤트를 건너뛰고 계속 발행

주문 A 완료 → 발행 실패

주문 A 취소 → 발행 성공

주문 A 완료 → 재시도 후 발행

수정: 주문별 순차 발행

주문 A 완료 → 발행 실패

주문 A 취소 → 선행 이벤트 재시도까지 보류

주문 B 이벤트 → 주문 A와 별도로 발행

라이선스가 아직 없어 취소를 처리하지 못한 뒤, 늦게 도착한 완료 이벤트로 라이선스가 지급될 수 있었습니다.

주문번호로 이벤트를 묶고 같은 주문은 앞선 발행의 응답을 확인한 뒤 다음 이벤트를 보냅니다. 다른 주문은 병렬로 발행합니다.

배치가 달라질 때도 보류가 유지돼야 했습니다

선행 이벤트만 재시도 시각을 늦추면 다음 조회에서 후행 이벤트만 선택될 수 있습니다. 같은 주문의 후행 이벤트에도 선행 이벤트의 재시도 시각을 적용했습니다.

발행 순서·대안 검토 · publishPreservingOrder / holdRemainder

ESD · 사례 2 / 검증 범위

배치 내·배치 간 순서와 다른 주문의 진행을 함께 검증

발행 순서 테스트와 백오프 조회 테스트는 각각 통과했지만, 두 조건이 만나는 동작은 확인하지 못했습니다. 발행 테스트의 대역에 실제 조회 시간 조건을 반영하자, 다음 배치에서 후행 이벤트가 앞서는 문제가 재현됐습니다.

상황	OutboxRelayTest의 판정
동일 주문의 정상 발행	적재 순서대로 발행합니다.
선행 이벤트 발행 실패	같은 주문의 후행 이벤트를 보류합니다.
다른 주문의 이벤트	실패한 주문과 별도로 계속 발행합니다.
재시도 시각 이전의 다음 조회	후행 이벤트가 선행 이벤트보다 먼저 발행되지 않습니다.
순서 때문에 보류된 후행 이벤트	발행을 시도하지 않았으므로 재시도 횟수를 차감하지 않습니다.
선행 이벤트가 DEAD로 전환	다음 회차에 해당 키의 후행 이벤트를 진행시킵니다.

재현한 조건을 회귀 테스트로 고정

재시도 시각 이전에 다시 조회하는 상황을 테스트에 넣었습니다. 후행 이벤트도 조회에서 보류되는지, 다른 주문은 계속 발행되는지 함께 확인했습니다.

보장 조건

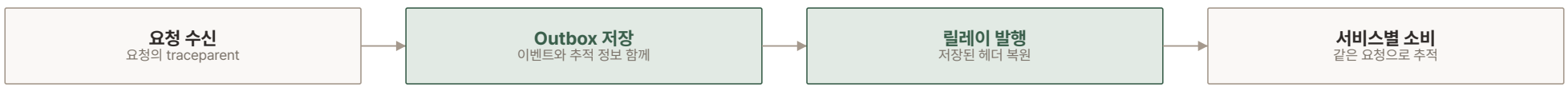
같은 주문을 릴레이 한 대가 처리하는 조건입니다. 다중 릴레이에서는 주문별 담당 분리가 추가로 필요합니다. DEAD 이벤트의 재투입 순서는 별도 복구 절차에서 다룹니다.

D-014 · 테스트가 놓친 조회 조건 · 순서·재시도 회귀 테스트 6개 · 다중 릴레이 확장 제약

ESD · 사례 3 / 관측과 원인 확인

Outbox 발행 때 원래 요청의 추적 정보가 사라짐

요청이 끝난 뒤 별도 스케줄러에서 Kafka로 발행하므로 요청 스레드의 추적 정보가 전달되지 않았습니다. 자동 계측을 적용한 뒤에도 원래 주문 요청 대신 릴레이 회차의 traceId가 기록됐습니다.



저장 위치를 정한 기준

이벤트와 추적 정보가 함께 커밋되도록 Outbox 행에 `traceparent`를 저장했습니다. 릴레이는 이 값을 Kafka 헤더에 복원합니다. 이벤트 본문에 추적 도구의 필드를 추가할 필요도 없었습니다.

발행 성공·재시도 시 저장된 값이 전달되는지, 추적 정보가 없는 이벤트에서는 헤더를 생략하는지 테스트했습니다.

추적 헤더 복원 코드 · 폴링 주기 비교 측정 원본 · 2026.08.10

연결된 트레이스에서 확인한 지연

> 결제 승인 요청
> → order · license · settlement: **+527ms**
> → license를 거친 후속 서비스: **+759ms**

6개 서비스를 같은 요청으로 연결한 뒤 서비스 사이의 대기 시간을 확인했습니다. Outbox의 1초 폴링 주기를 줄여 효과를 측정했습니다.

ESD · 처리량과 종단 지연 / 별도 실험

배치 사이 대기와 새 이벤트의 발행 대기를 각각 줄임

사전 적재한 이벤트의 발행 처리량과 결제 요청 후 이용 권한 확인 지연을 별도 실험으로 비교했습니다. 조건마다 두 번 측정했습니다.

별도 실험 · 배치 사이의 대기 제거

연속 발행 배치 수	1회	2회	평균
1개	180.7	180.7	180.7
10개	975.6	921.7	**948.7**

단위: events/s · 2만 건 사전 적재 · 배치 200건.
Outbox에 미발행 이벤트가 남아 있어도 배치마다 1초씩 기다렸습니다. 폴링 주기는 유지하고 최대 10개 배치를 연속 발행했습니다. 모든 회차의 실패·DEAD는 0건이었습니다.

180.7 → 948.7건/초

릴레이 발행 처리량 · 회차 평균

사례 3 · 다음 조회까지 대기 1초 → 200ms

주기	1회 p95	2회 p95	회차 평균
1000ms	1.30초	1.06초	1.18초
200ms	563ms	561ms	**562ms**

k6 5 VU·2분 · 승인 콜백 요청 → 라이브러리 API의 이용 권한 확인.
완료 건수는 회차별 252/198건에서 285/286건으로 늘었고, 네 회차 모두 성공률 100%였습니다.

1.18 → 0.56초

요청부터 이용 권한 확인까지 · 회차별 p95 평균

폴링 주기를 줄이면서 증가한 DB 부하

MySQL CPU 회차 평균은 **7.3% → 8.8%**였습니다. 이 환경에서 지연 감소와 DB 부하를 확인하고 200ms를 기본값으로 적용했습니다.

측정 범위

지급 여부를 0.5초마다 조회해 더 짧은 지연을 구분하기 어렵습니다. 중앙값의 개선폭과 더 큰 부하의 처리 한계는 이 측정으로 확인할 수 없었습니다.

2026.08.10 측정 표 · 측정 시작·종료 코드 · 기준 변경: `7574ea2` · 과거 측정 환경의 결과입니다.

SNG · 개요

게임 서버와 웹 상점의 구매·보상 처리 구현

개인 프로젝트 · 2026.06–07 · Kotlin/Spring 기반 게임 서버와 웹 상점입니다. 웹 상점의 마일리지 구매·보상 전달과 게임 서버의 시즌 순위 저장·보상 우편 지급을 구현했습니다.

구간	처리하는 업무	검증한 문제
웹 상점	구매 한도 확인 · 주문 저장 · 마일리지 차감	동시 구매의 커넥션 대기 · 잔액과 원장 정합성
게임 서버	시즌 순위 저장 · 보상 우편 지급 · 정산 재개	실패 후 진행 상태 보존 · 재개 시 중복 지급

동시 구매가 멈춘 원인 분석

주문 트랜잭션 안에서 초기화용 트랜잭션을 따로 열어 추가 커넥션을 요구했습니다. 동시 요청이 풀을 점유하면 모두 다음 커넥션을 기다렸습니다.
초기화 경계를 바꾸고 MySQL에서 검증했습니다. 12쪽 · 13쪽

재개 테스트가 놓친 롤백

목 테스트는 재개 상태를 직접 만들어 통과했습니다. 실제 실패에서는 그 상태도 롤백돼, 단계별 커밋과 지급 상태로 재개하도록 바꿨습니다.
실제 트랜잭션을 사용하는 H2 테스트로 상태 보존과 재지급 여부를 확인했습니다. 14쪽

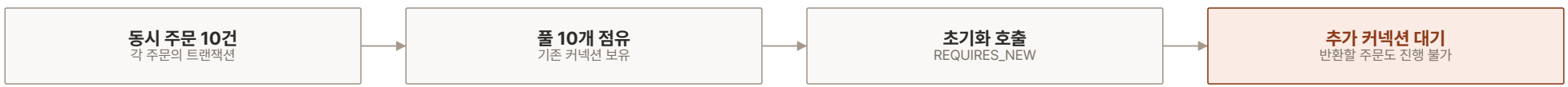
Kotlin · Spring Boot · JPA · MySQL · Redis Streams · Docker · JUnit 5 · Testcontainers

비공개 저장소 · 기준 `1292702` · 다음 세 쪽에 핵심 코드, 수정 이력과 검증 조건을 수록했습니다.

SNG · 동시 구매 / 실패와 원인

동시 구매가 모두 추가 DB 커넥션을 기다림

CI의 마일리지 동시 구매 테스트가 실패했습니다. 재현 기록에서는 요청 10건이 모두 커넥션 획득 중 대기했습니다. 주문 트랜잭션이 커넥션을 보유한 채, 초기화용 별도 트랜잭션에 필요한 커넥션을 추가로 요구하고 있었습니다.



변경 전 호출 구조 · 요약

```
// PlaceOrder: 주문 트랜잭션
@Transactional
fun place(...) { ... ensureAccount(...) }

// 다른 빈에서 초기화용 트랜잭션 생성
@Transactional(propagation = REQUIRES_NEW)
fun ensureAccount(...) { ... }
```

수정 `295ee701`의 재현 기록: 성공 0건 · Interrupted during connection acquisition.
수정 전 `338ef8f` CI에서 해당 동시 구매 테스트 실패 확인 · PlaceOrder / MileageWriter / PurchaseLimitGuard.

초기화를 분리했던 이유

계정·구매 한도 행을 동시에 만들 때의 중복 키 예외가 주문 전체를 롤백시키지 않도록 초기화 트랜잭션을 분리했습니다. 그러나 바깥 주문의 커넥션은 그대로 점유하고 있었습니다.

실패 격리와 자원 점유를 함께 확인

초기화의 중복을 예외 없이 처리하면 주문 트랜잭션 안에서 실행할 수 있습니다. 별도 커넥션을 요구하던 초기화 경계를 제거하는 방향으로 수정했습니다.

SNG · 동시 구매 / 수정과 검증

초기화를 주문 트랜잭션에 통합하고 MySQL에서 검증

계정·구매 한도 초기화를 주문 트랜잭션에 통합했습니다. 중복 생성은 `INSERT IGNORE`로 처리해 초기화용 추가 커넥션을 요구하지 않게 했습니다.

계정 초기화 · MileageWriter 발췌

```
@Transactional
fun ensureAccount(accountId: Long) {
    if (accountMileageRepository
        .existsById(accountId)) return
    accountMileageRepository.insertIfAbsent(
        accountId, LocalDateTime.now())
}
```

기존 행의 잠금 경합도 보완

기존 행에도 삽입을 시도하자 공유 잠금 이후 차감에 필요한 잠금으로 전환하면서 경합했습니다. 이미 있는 행은 삽입을 건너뛰도록 보완했습니다. 구매 한도 초기화에도 같은 방식을 적용했습니다.

ConcurrencyIntegrationTest.mileageSpendCannotOverdraw · `338ef8f` 실패 → `295ee701` 통과.
변경 범위는 주문 내부의 초기화입니다. 배치 적립 등 별도 실패 격리가 필요한 경로의 REQUIRES_NEW는 유지했습니다.

동시 요청과 잔액·원장을 함께 확인

잔액 300 · 상품 금액 100 · 동시 요청 10건. 이 조건에서 성공해야 하는 구매는 3건입니다.

검증 항목	수정 후
구매 성공	**3건**
최종 잔액	**0**
잔액과 원장 합계	**일치**

실제 DB를 사용하는 통합 테스트

Testcontainers의 MySQL 8.0과 Flyway 스키마로 검증했습니다. 수정 전 CI에서 실패한 테스트가 수정 후 통과했습니다.

재개 테스트가 놓친 상태 롤백을 트랜잭션 경계로 해결

초기 정산은 전체를 한 트랜잭션으로 감쌌습니다. 목 테스트는 `SETTLING` 상태를 직접 만들어 재개를 확인했지만, 실제 실패 시에는 그 상태와 지급 기록도 롤백돼 이어갈 지점이 남지 않았습니다.

진행은 보존하고 지급·완료 상태는 함께 저장

전체 정산의 트랜잭션을 제거해 단계별로 커밋했습니다. 200건씩 보상을 지급하며, 우편과 `SENT` 상태를 같은 트랜잭션에 저장했습니다.

```
// 청크 처리 구조 요약
checkpointTx.commit {
    // PENDING 첫 페이지 200건 잠금 조회
    // 보상 우편 생성 + SENT 상태 저장
}
```

완료 행은 다음 조회에서 빠지므로 항상 첫 페이지를 조회합니다. 실패한 청크는 함께 롤백하고, 재개 시 남은 `PENDING`만 처리합니다.

H2 · 1만 건 · 7,001번째 보상 계산에 예외 주입

확인 항목	단일 트랜잭션	청크 커밋
중단 후 지급·우편	0건	**7,000건**
중단 후 시즌 상태	ACTIVE	**SETTLING**
재개 후 추가 지급	비교 대상 아님	**3,000건**
재개 후 총 우편	비교 대상 아님	**10,000건**

목 테스트를 실제 트랜잭션을 사용하는 H2 테스트로 바꿨습니다. 비교용 단일 트랜잭션을 구성해 같은 예외 조건에서 상태 보존과 재지급 여부를 확인했습니다.

실제 변경 이력: `b44f7e4` 단일 트랜잭션 → `e83f23b` 단계별 커밋 · 비교 테스트 추가 `23dda60`.
SettleSeasonBeforeAfterTest · 기준 `1292702` · H2 예외 주입 검증이며 프로세스 강제 종료·MySQL 잠금 경합 검증과 구분합니다.

JobMate · 담당 기능과 처리 구조

S3 직접 업로드·별도 영상 변환·상태 조회 구현

팀 5명 · 2025.09–10 · 프로그래머스 백엔드 데브코스 프로젝트입니다. IT 진로 탐색·멘토링 서비스의 영상 API와 변환 워커를 담당했습니다. 프론트엔드 개발자에게 API 구현 방식을 공유하며 업로드·상태 조회 기능을 연동했습니다.



담당 구현	처리하는 작업	다음 단계로 전달하는 값
Spring API	권한·확장자 확인, 영상 레코드 관리, 업로드·재생 URL 발급	UUID · 업로드 URL · 화질별 상태
HTTP 브리지	SNS 구독 확인, S3 객체 생성 메시지 파싱, Kafka 발행	bucket · object key → s3-events
Python 워커	ffprobe 검사, 1080p·720p·480p 변환, 결과 업로드	transcoding-status에 화질별 상태 발행

직접 업로드와 별도 워커를 둔 이유

파일 전송과 CPU를 사용하는 변환 작업을 API 요청에서 분리했습니다. 클라이언트는 S3에 직접 업로드하고, 별도 컨테이너의 워커가 변환합니다.

화질별 변환 상태 관리

워커가 대기·진행·완료·실패 상태를 Kafka로 전달하면 Spring에서 UUID에 연결해 저장합니다. 클라이언트가 변환 완료 여부를 조회하고 재생 URL을 요청할 수 있습니다.

Java 21 · Spring Boot · AWS S3/SNS · Kafka · Python · ffmpeg · Docker

영상 API · HTTP 브리지 · 변환 워커 · 컨테이너 구성

업로드 완료 전 뉴스 제출 오류 수정과 결과물 재입력 방지

업로드 완료 전 뉴스 제출 시 404

영상 업로드가 끝나기 전에 뉴스를 제출하면, 참조할 영상 레코드가 없어 404가 발생했습니다. 기존에는 워커의 상태 메시지를 받은 뒤 없는 레코드를 생성했습니다.

URL 발급 시 레코드 생성
UUID 생성
→ 업로드 URL 발급
→ READY_TO_UPLOAD 상태 저장
→ 업로드·변환 결과로 상태 갱신

업로드 URL 발급 시 영상 레코드를 먼저 생성했습니다. 뉴스는 업로드 완료 전에도 UUID로 영상을 참조하고, 이후 상태 메시지로 레코드를 갱신합니다.

변환 결과물을 입력 대상에서 제외

원본과 변환 결과물을 같은 버킷에 저장하도록 바꾸면서, 결과물이 다시 처리되지 않도록 경로를 구분했습니다.

```
# 변환 결과물은 다운로드 전에 제외
if key.startswith("transcoded/"):
    continue
```

원본은 `videos/`, 결과물은 `transcoded/` 경로를 사용했습니다. 결과물 이벤트는 다운로드와 변환 대상에서 제외했습니다. 같은 원본 이벤트의 중복 전달에 대한 멱등 처리는 추가로 필요합니다.

URL 발급·권한·상태 갱신의 모듈 테스트가 있습니다. URL 발급 테스트는 FileManager를 모킹하므로, 업로드 완료 전 뉴스 제출 오류의 회귀 검증은 포함하지 않습니다. 워커를 포함한 종단 검증은 별도로 필요합니다.

업로드 완료 전 제출 실패 이슈 · 레코드 생성 시점 수정 · 결과물 재입력 제외 · 영상 API 테스트

코드 · 테스트 · 실험 기록

사례별 근거와 검증 기준점

사례	근거 자료 · 링크	기준
ESD 오환불	D-027·D-028 원인·수정 · 장애 실험 환불·보류 조건 테스트	`a525243`
ESD 발행 순서	보장 범위와 대안 · OutboxRelayTest	`a525243`
ESD 추적·성능	traceparent 복원 · 폴링 주기별 원본 측정 표	측정 2026.08.10 원본 `17786a5`
ESD API 데모	실행 절차·빈 환경 두 회차 기록	실행 `7873593` 기록 2026.09.30
SNG 동시 구매	MileageWriter · PurchaseLimitGuard ConcurrencyIntegrationTest · MySQL 통합 테스트 CI	`338ef8f` → `295ee701` 12–13쪽 수록
SNG 정산	단일 트랜잭션 제거 · 목 테스트의 재개 상태 주입 교체 SettleSeasonBeforeAfterTest · H2 예외 주입 비교	`e83f23b` · `1292702` 14쪽 수록
JobMate 영상	URL·상태 관리 · 변환 워커 영상 테스트 · 상태 연동 커밋 · 브리지 수정	`a4e2d35`

SNG는 비공개 저장소이므로 핵심 코드와 검증 조건을 12–14쪽에 수록했습니다. 구매는 MySQL, 정산 비교는 H2입니다. ESD의 장애·성능 실험과 2026.09.30 API 데모는 각각 별도로 실행한 기록입니다.

대표 구현 · 다음 작업

원인을 좁히고 수정 결과를 검증한 경험

프로젝트	구현한 변경과 결과	사례
ESD	이벤트 추적을 연결해 발행 대기를 줄였습니다. 장애 범위를 나눠 잘못된 환불 원인을 찾고 판단 조건을 수정했습니다.	5쪽 · 9쪽
SNG	동시 구매가 추가 커넥션을 기다리는 구조를 찾아 초기화를 주문 트랜잭션에 통합하고, MySQL에서 잔액·원장 정합성을 검증했습니다.	12쪽 · 13쪽
JobMate	영상 API를 프론트엔드와 연동하고, 업로드 완료 전 뉴스 제출 오류를 수정했습니다. 변환 결과물은 입력에서 제외했습니다.	16쪽

현재 MVP · 단계별 구현과 검증

모듈별 책임·계약·완료 조건을 정리해 11개 모듈을 재작성했습니다. 모의 PG를 포함한 실제 프로세스 검증 32단계를 통과했습니다. OCI의 방식별 3회 비교에서 이미지 생성·게시 시간 중앙값을 65.4% 줄였습니다.

문서 기준 2026.10.24 · 현재 MVP 검증 · 이미지 생성·게시 비교 · 목차로 이동

연락처

신지섭 · Java·Kotlin / Spring 백엔드
wltjq1203@icloud.com
github.com/tlswltjq